

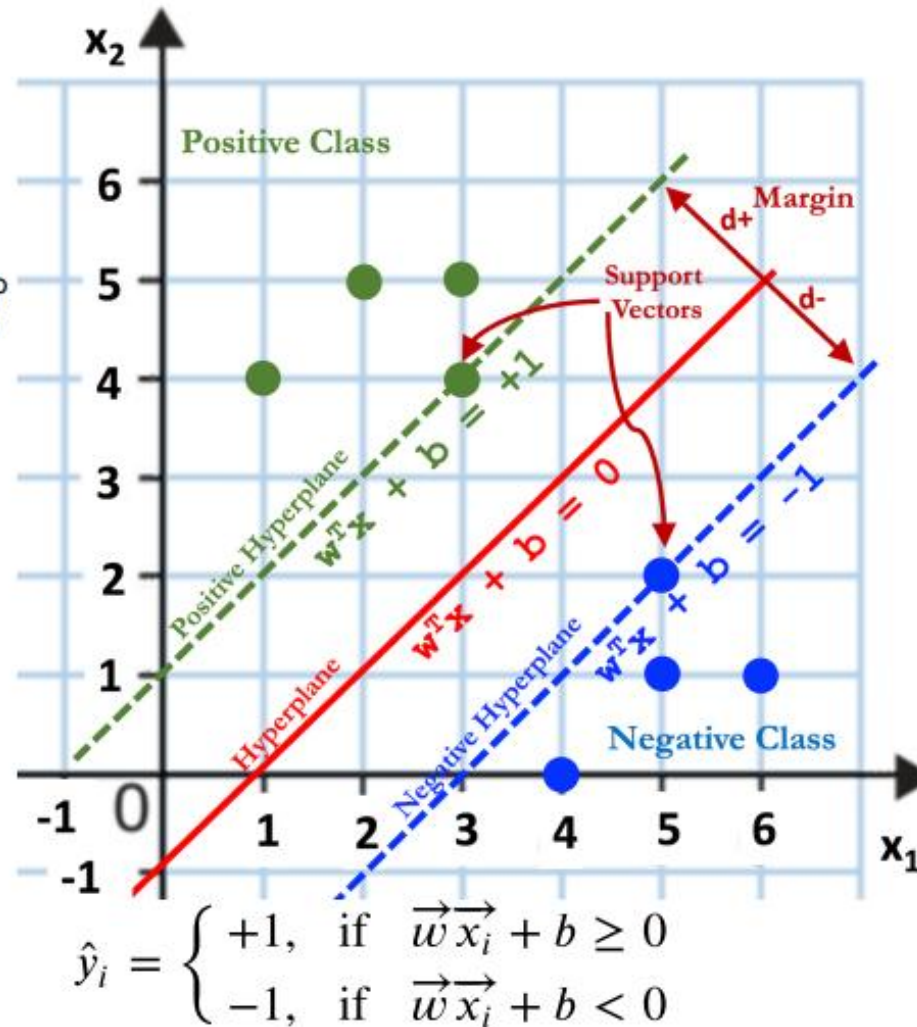
Support Vector Machine

Lecture 21

Recap

1. Recap of Hard Margin SVM

- General Rule to Classify a New Data Point:



Hard Margin SVM

- **Constraint for Hard SVM:**

- We need to maximize the margin, such that

- NO training point is below the Positive Hyperplane :

$$\vec{w} \cdot \vec{x} + b \geq 1$$

- NO training point is above the Negative Hyperplane :

$$\vec{w} \cdot \vec{x} + b < -1$$

- Simplify the above two equation to one, assuming that the label of all the positive training points is $+1$ and the label of all the negative training points is -1 :

Hard Margin SVM

$$y_i(\vec{w} \vec{x}_i + b) \geq 1$$

$$\operatorname{argmax}_{(w^*, b^*)} \quad \frac{2}{\|w\|}$$

$$\operatorname{argmax}_{(w^*, b^*)} \quad \frac{1}{\|w\|}$$

$$\operatorname{argmin}_{(w^*, b^*)} \quad \|w\|$$

$$\operatorname{argmin}_{(w^*, b^*)} \quad \frac{1}{2} \|w\|^2,$$

Langrange Multipliers

$$L = \frac{1}{2} \|\vec{w}\|^2 - \sum \alpha_i [y_i (\vec{w} \cdot \vec{x}_i + b) - 1]$$

$$\frac{\partial L}{\partial \vec{w}} = \vec{w} - \sum \alpha_i y_i \vec{x}_i$$

$$\frac{\partial L}{\partial b} = - \sum \alpha_i y_i$$

$$\vec{w} - \sum \alpha_i y_i \vec{x}_i = 0$$

$$- \sum \alpha_i y_i = 0$$

$$\vec{w} = \sum \alpha_i y_i \vec{x}_i$$

$$\sum \alpha_i y_i = 0$$

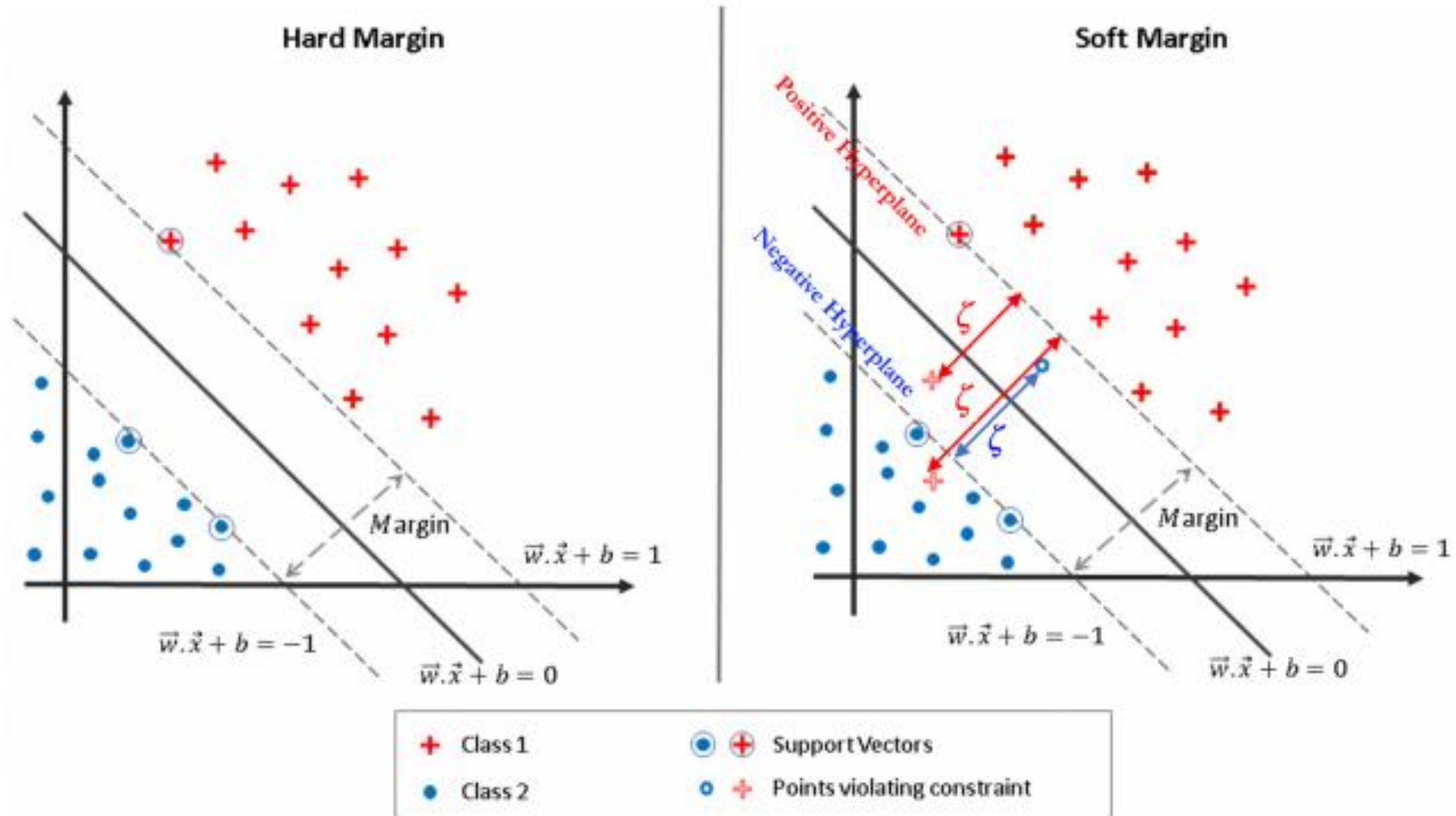
$$L = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$$

Soft Margin SVM

- The **hard margin** is the decision boundary that maximizes the margin and separates the classes perfectly. This works well when the data is linearly separable, but it is sensitive to outliers and noise.
- The **soft margin** allows for some misclassification of the data points in order to maximize the margin. This is useful when the data is not linearly separable and there are some outliers or noise in the data.

In **Soft Margin** we allow SVM to make a certain number of mistakes and keep margin as wide as possible so that other points can still be classified correctly.

Soft Margin SVM



Soft Margin SVM

$$\operatorname{argmin}_{(w^*, b^*)} \frac{1}{2} \|w\|^2, \quad \text{such that} \quad y_i(\vec{w} \vec{x}_i + b) \geq 1$$

$$\operatorname{argmin}_{(w^*, b^*)} \frac{1}{2} \|w\|^2 + C \sum \zeta_i, \quad \text{such that} \quad y_i(\vec{w} \vec{x}_i + b) \geq 1 - \zeta_i$$

Slack Variable

ζ_i :

- For every data point x_i , we introduce a slack variable ζ_i . The value of ζ_i is the distance of x_i from the corresponding class's margin if x_i is on the wrong side of the margin, otherwise zero.
- For all positive points above the positive hyperplane, the ζ value will be zero.
- For all negative points below the negative hyperplane, the ζ value will be zero.
- For a positive data point below the decision boundary the corresponding ζ value will be greater than 1, while for a positive data point that is on the correct side of the decision boundary, but within the margin will have a ζ value between 0 and 1.
- For a negative data point above the decision boundary the corresponding ζ value will be greater than 1, while for a negative data point that is on the correct side of the decision boundary, but within the margin will have a ζ value between 0 and 1.
- Thus the points that are far away from the margin on the wrong side would get more penalty.

Regularization

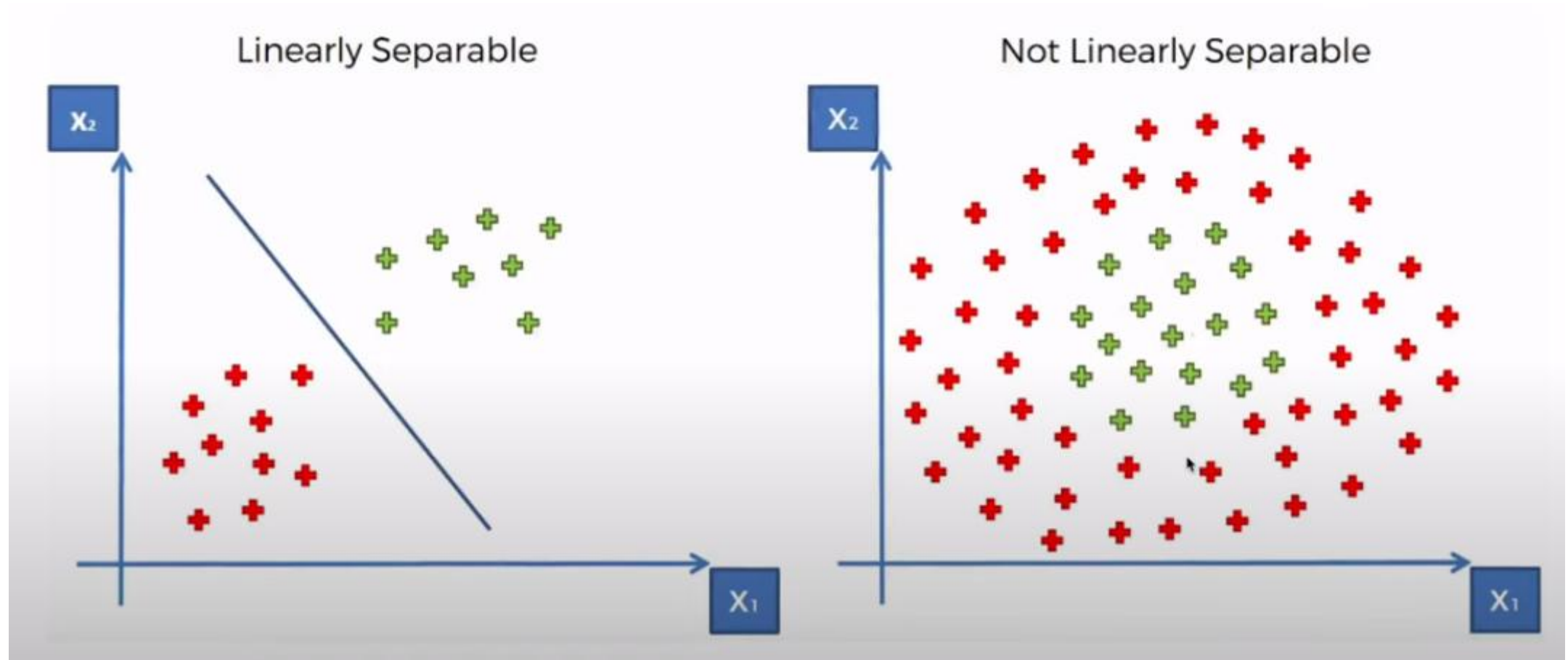
- The regularization hyperparameter C controls the trade-off between maximizing the margin and minimizing the misclassification.
- A very small value of C , makes the second term of above expression small, thus the model will ignore classification errors and will focus on increasing the margin.
- A large value of C will not concentrate on maximizing the margin, rather will focus on reducing the number of miss-classified points.

Classifying Non-Linear Data using SVM

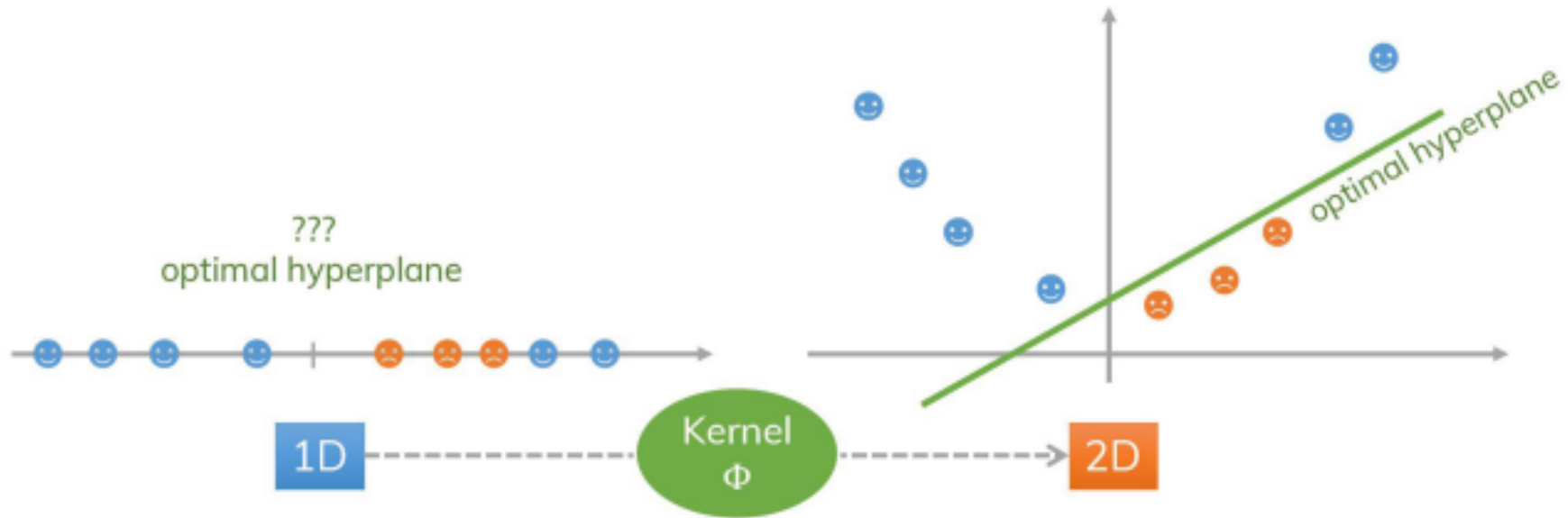
Linear VS Non Linear

- **Linear SVM:** When the data is perfectly linearly separable only then we can use Linear SVM. Perfectly linearly separable means that the data points can be classified into 2 classes by using a single straight line(if 2D).
- **Non-Linear SVM:** When the data is not linearly separable then we can use Non-Linear SVM, which means when the data points cannot be separated into 2 classes by using a straight line (if 2D) then we use some advanced techniques like kernel tricks to classify them. In most real-world applications we do not find linearly separable datapoints hence we use kernel trick to solve them.

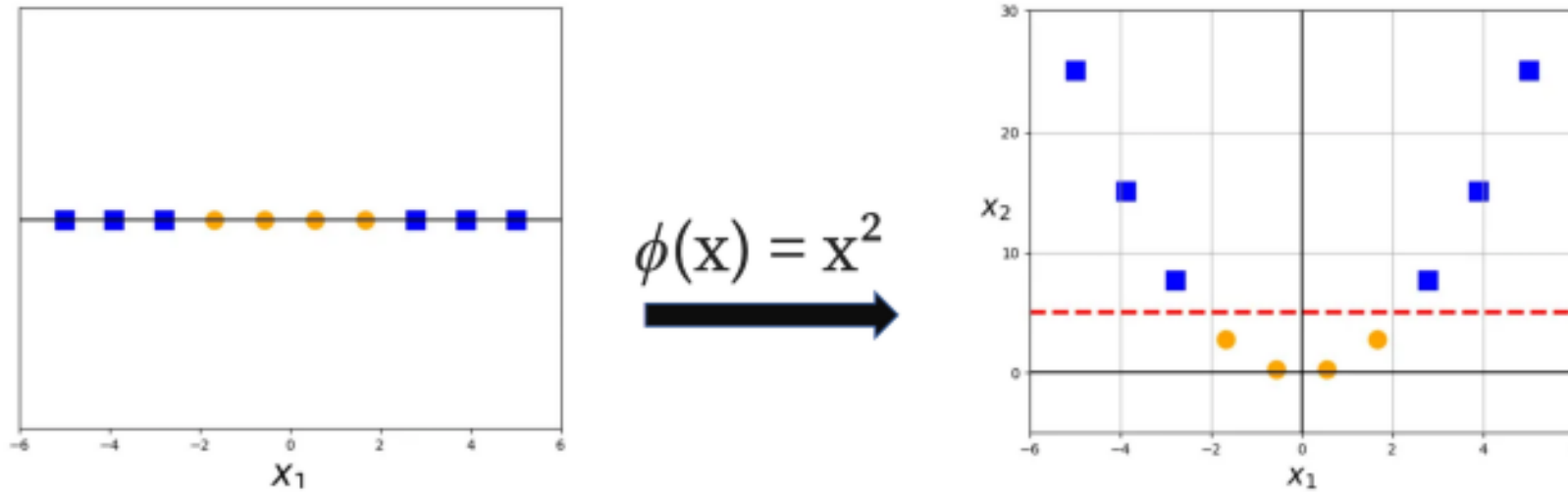
Linear VS Non Linear



Mapping a 1D Dataset to 2D to make it Linearly Separable

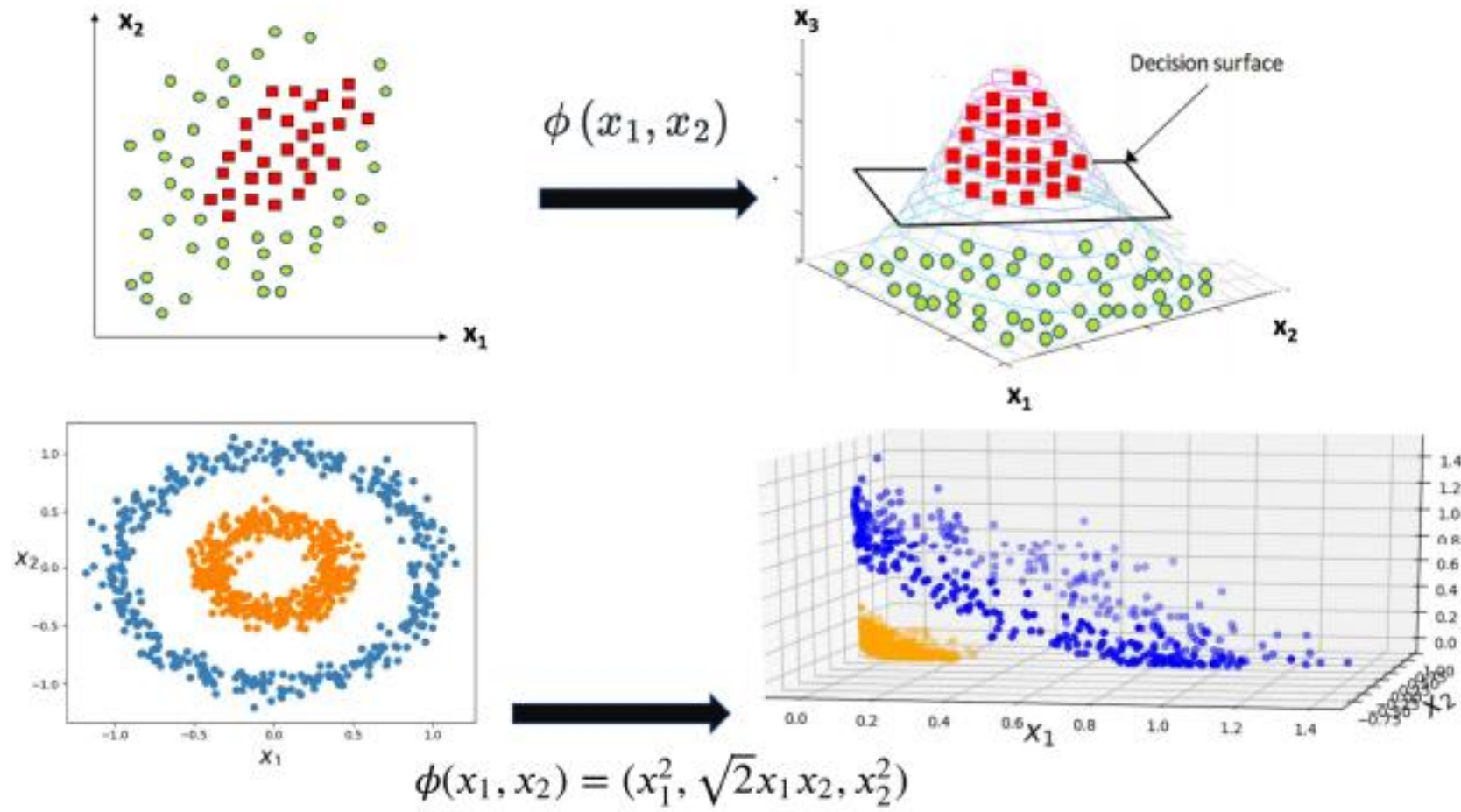


Mapping a 1D Dataset to 2D to make it Linearly Separable



- In 1-dimension, this data is not linearly separable, but after applying the transformation $\phi(x) = x^2$ and adding this second dimension to our feature space, the classes become linearly separable.

Mapping a 2D dataset to 3D to make it Linearly



(1, 2) will be mapped to $(1, 2\sqrt{2}, 4)$ and (3, 4) will
be mapped to $(9, 12\sqrt{2}, 16)$

Limitations

- Which transformation function?
- Computation

SVM Kernel Trick

A kernel function is a mathematical function that takes as its inputs, vectors in the original (lower dimensional) space, and return the dot product of the transformed vectors in the feature (higher dimensional) space.

SVM Kernel Trick

A kernel function is a mathematical function that takes as its inputs, vectors in the original (lower dimensional) space, and return the dot product of the transformed vectors in the feature (higher dimensional) space.

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel
- Sigmoid Kernel Laplace RBF Kernel
- Hyperbolic Tangent Kernel Bessel
- Function Kernel
- Anova RBF Kernel
- Linear Spline Kernel

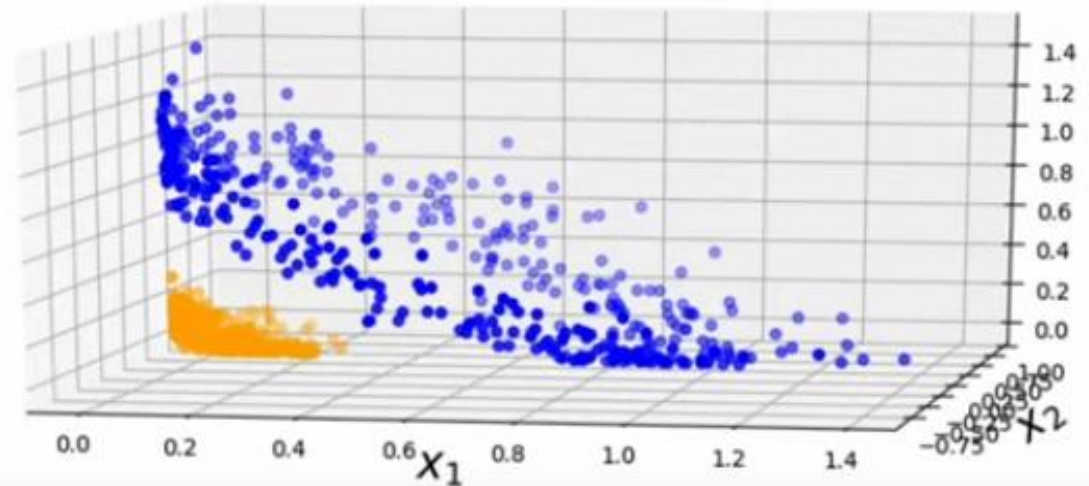
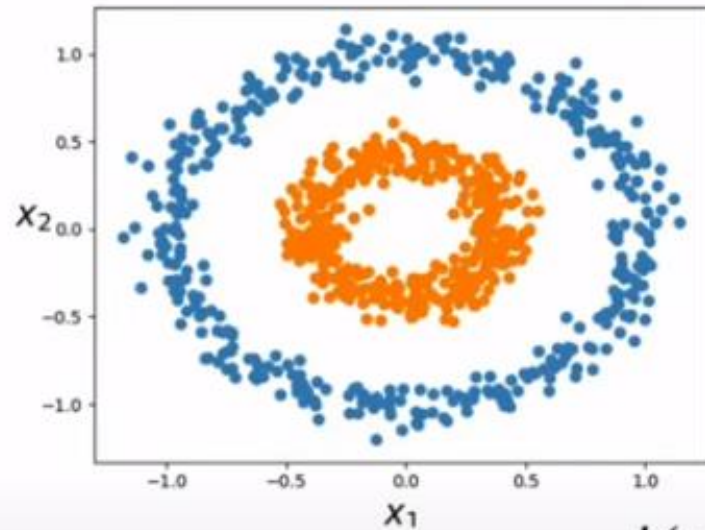
Linear Kernel

- The most simplest of all is Linear Kernel, usually one dimensional in nature.
- It proves to be the best function when there are lots of features.
- The linear kernel is mostly preferred for text-classification problems as most of these kinds of classification problems can be linearly separated.

$$K(\vec{x}_1, \vec{x}_2) = \phi(\vec{x}_1) \cdot \phi(\vec{x}_2) = \vec{x}_1^T \cdot \vec{x}_2$$

Polynomial Kernel

$$K(\vec{x}_1, \vec{x}_2) = (\vec{x}_1^T \cdot \vec{x}_2 + c)^d$$



$$\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Use Mapping Function:

- Apply mapping function of 1st data point: $\phi(1, 2) = (1, 2\sqrt{2}, 4)$
- Apply mapping function of 2nd data point: $\phi(3, 4) = (9, 12\sqrt{2}, 16)$
- Take dot product: $\phi(1, 2) \cdot \phi(3, 4) = (1, 2\sqrt{2}, 4) \cdot (9, 12\sqrt{2}, 16) = (1 * 9 + 2\sqrt{2} * 12\sqrt{2} + 4 * 16) = 121$

Polynomial Kernel

$$\begin{aligned}\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^T \cdot \mathbf{b})^2\end{aligned}$$

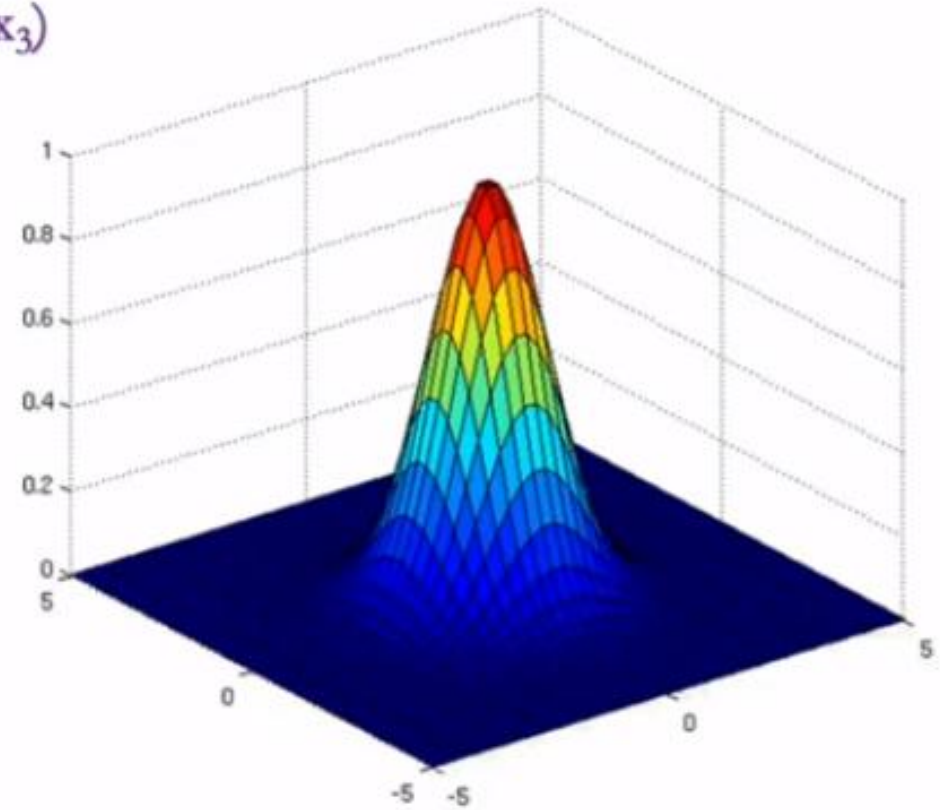
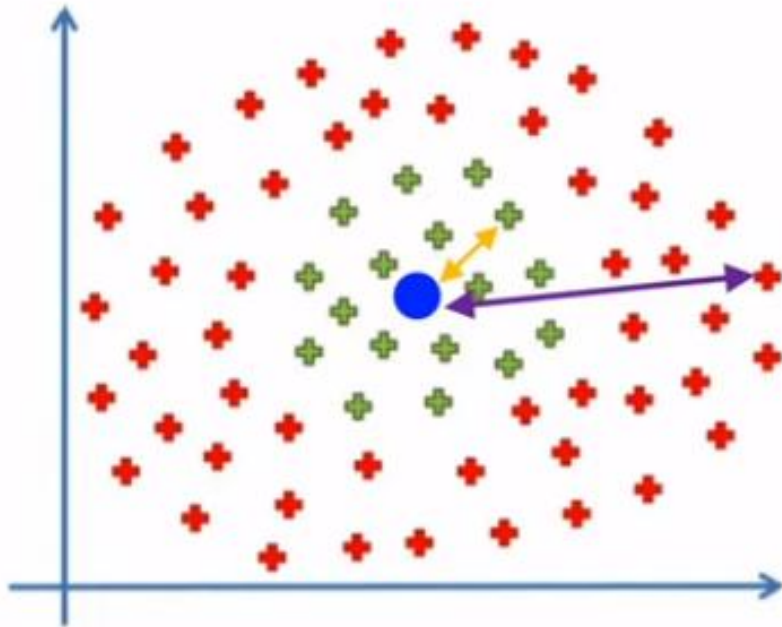
Use Kernel Trick:

$$\bullet K(a, b) = (a^T \cdot b)^2 = \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot [3 \ 4] \right)^2 = (1 * 3 + 2 * 4)^2 = 121$$

Radial Basis Function (RBF) Kernel

$$K(\vec{x}, \vec{l}^i) = e^{-\gamma \|\mathbf{x} - \mathbf{l}^i\|^2} = e^{-\frac{\|\mathbf{x} - \mathbf{l}^i\|^2}{2\sigma^2}}$$

$$d_{12} < d_{13} \Rightarrow K(\mathbf{x}_1, \mathbf{x}_2) > K(\mathbf{x}_1, \mathbf{x}_3)$$



Radial Basis Function (RBF) Kernel

- **Example 1:** Consider two data points $x_1 = (1, 2)$ and $l = (1, 2)$ with $\sigma = 1$. Calculate the value of RBF kernel

$$K(x_1, x_2) = e^{-\frac{(1-1)^2 + (2-2)^2}{2 \cdot 1^2}} = e^0 = 1.0$$

- **Example 2:** Consider two data points $x_1 = (1, 2)$ and $l = (2, 3)$ with $\sigma = 1$. Calculate the value of RBF kernel

$$K(x_1, x_2) = e^{-\frac{(2-1)^2 + (3-2)^2}{2 \cdot 1^2}} = e^{-1} = 0.368$$

Thank You..!